16 October 2025

CODE

```
--Q1. Display all columns for all transactions.

--Expected output: All columns

SELECT*

FROM practical1.dataset.retail_sales
```

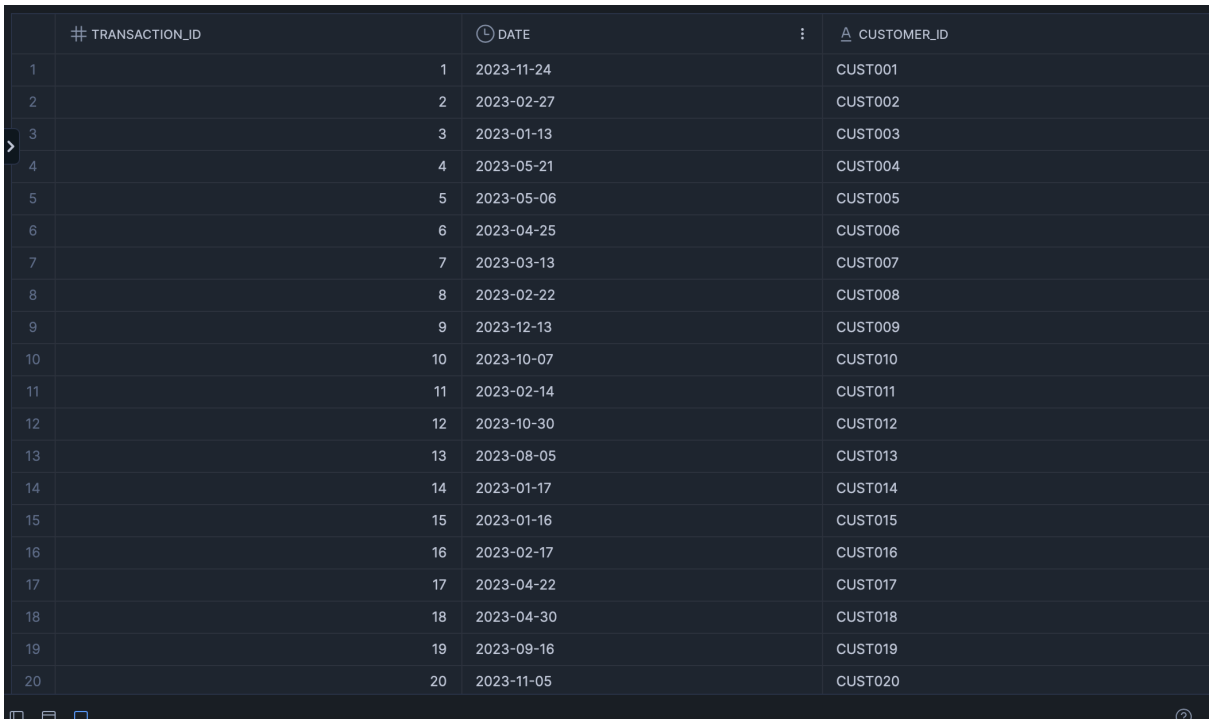
OUTPUT

	_ID	DATE	CUSTOMER_ID	GENDE	AGE	PRODUCT_CATEGORY	QUANTITY	PRICE_PER_UNIT	TOTAL_AMOUNT
1	1	2023-11-24	CUST001	Male	34	Beauty	3	50	150
2	2	2023-02-27	CUST002	Female	26	Clothing	2	500	1000
3	3	2023-01-13	CUST003	Male	50	Electronics	1	30	30
4	4	2023-05-21	CUST004	Male	37	Clothing	1	500	500
5	5	2023-05-06	CUST005	Male	30	Beauty	2	50	100
6	6	2023-04-25	CUST006	Female	45	Beauty	1	30	30
7	7	2023-03-13	CUST007	Male	46	Clothing	2	25	50
8	8	2023-02-22	CUST008	Male	30	Electronics	4	25	100
9	9	2023-12-13	CUST009	Male	63	Electronics	2	300	600
10	10	2023-10-07	CUST010	Female	52	Clothing	4	50	200
11	11	2023-02-14	CUST011	Male	23	Clothing	2	50	100
12	12	2023-10-30	CUST012	Male	35	Beauty	3	25	75
13	13	2023-08-05	CUST013	Male	22	Electronics	3	500	1500
14	14	2023-01-17	CUST014	Male	64	Clothing	4	30	120
15	15	2023-01-16	CUST015	Female	42	Electronics	4	500	2000
16	16	2023-02-17	CUST016	Male	19	Clothing	3	500	1500
17	17	2023-04-22	CUST017	Female	27	Clothing	4	25	100
18	18	2023-04-30	CUST018	Female	47	Electronics	2	25	50
19	19	2023-09-16	CUST019	Female	62	Clothing	2	25	50
20	20	2023-11-05	CUST020	Male	22	Clothing	3	300	900

CODE

```
--Q2. Display only the Transaction ID, Date, and Customer ID for all records.
--Expected output: Transaction ID, Date, Customer ID
SELECT transaction_id,
    date,
    customer_id
FROM practical1.dataset.retail_sales;
```

OUTPUT

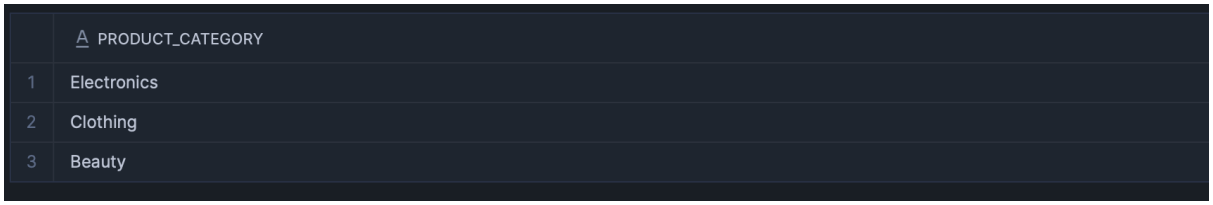


	TRANSACTION_ID	DATE	CUSTOMER_ID
1	1	2023-11-24	CUST001
2	2	2023-02-27	CUST002
3	3	2023-01-13	CUST003
4	4	2023-05-21	CUST004
5	5	2023-05-06	CUST005
6	6	2023-04-25	CUST006
7	7	2023-03-13	CUST007
8	8	2023-02-22	CUST008
9	9	2023-12-13	CUST009
10	10	2023-10-07	CUST010
11	11	2023-02-14	CUST011
12	12	2023-10-30	CUST012
13	13	2023-08-05	CUST013
14	14	2023-01-17	CUST014
15	15	2023-01-16	CUST015
16	16	2023-02-17	CUST016
17	17	2023-04-22	CUST017
18	18	2023-04-30	CUST018
19	19	2023-09-16	CUST019
20	20	2023-11-05	CUST020

CODE

```
--2. SELECT DISTINCT Statement
--Q3. Display all the distinct product categories in the dataset.
--Expected output: Product Category
SELECT DISTINCT product_category
FROM practical1.dataset.retail_sales;
```

OUTPUT

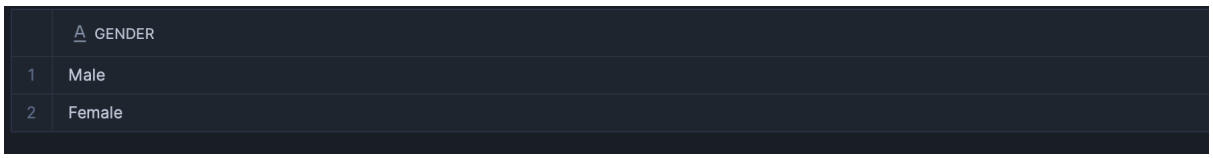


	PRODUCT_CATEGORY
1	Electronics
2	Clothing
3	Beauty

CODE

```
--Q4. Display all the distinct gender values in the dataset.

--Expected output: Gender

SELECT DISTINCT Gender

FROM practical1.dataset.retail_sales;
```

OUTPUT

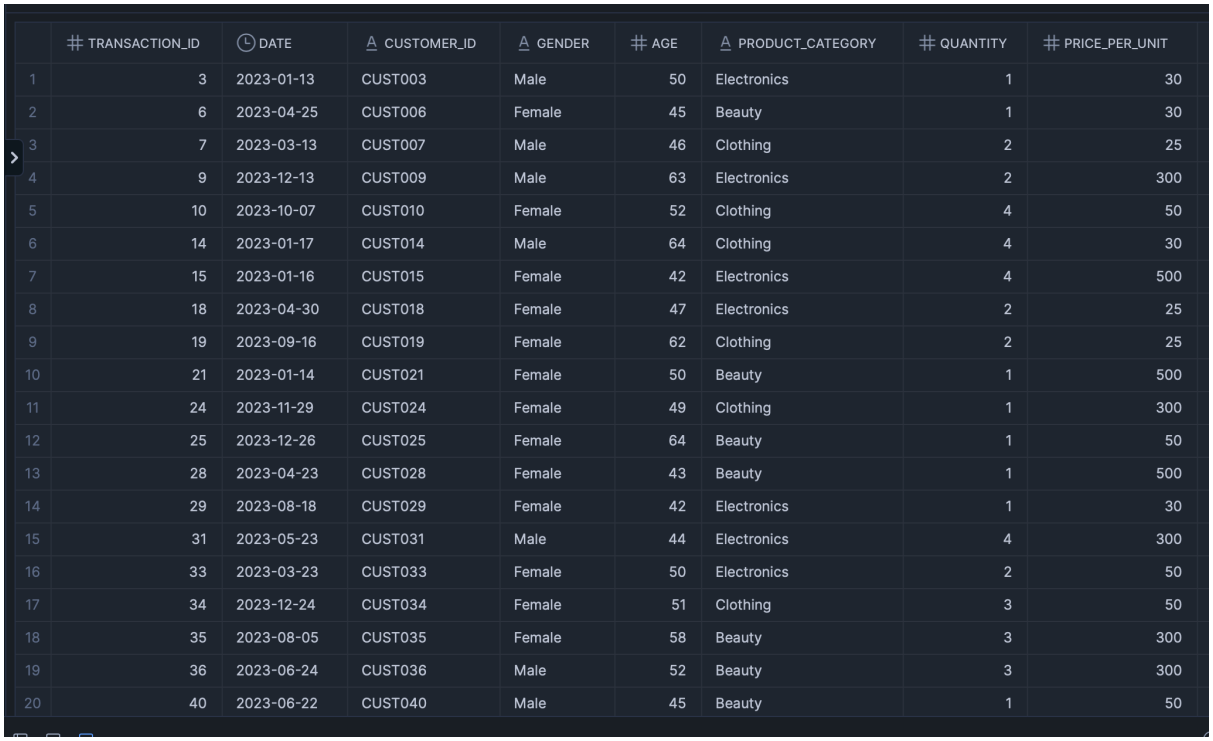


	GENDER
1	Male
2	Female

CODE

```
--3. WHERE Clause

--Q5. Display all transactions where the Age is greater than 40.

--Expected output: All columns

SELECT *

FROM practical1.dataset.retail_sales

WHERE Age > 40;
```

OUTPUT



	TRANSACTION_ID	DATE	CUSTOMER_ID	GENDER	AGE	PRODUCT_CATEGORY	QUANTITY	PRICE_PER_UNIT
1	3	2023-01-13	CUST003	Male	50	Electronics	1	30
2	6	2023-04-25	CUST006	Female	45	Beauty	1	30
3	7	2023-03-13	CUST007	Male	46	Clothing	2	25
4	9	2023-12-13	CUST009	Male	63	Electronics	2	300
5	10	2023-10-07	CUST010	Female	52	Clothing	4	50
6	14	2023-01-17	CUST014	Male	64	Clothing	4	30
7	15	2023-01-16	CUST015	Female	42	Electronics	4	500
8	18	2023-04-30	CUST018	Female	47	Electronics	2	25
9	19	2023-09-16	CUST019	Female	62	Clothing	2	25
10	21	2023-01-14	CUST021	Female	50	Beauty	1	500
11	24	2023-11-29	CUST024	Female	49	Clothing	1	300
12	25	2023-12-26	CUST025	Female	64	Beauty	1	50
13	28	2023-04-23	CUST028	Female	43	Beauty	1	500
14	29	2023-08-18	CUST029	Female	42	Electronics	1	30
15	31	2023-05-23	CUST031	Male	44	Electronics	4	300
16	33	2023-03-23	CUST033	Female	50	Electronics	2	50
17	34	2023-12-24	CUST034	Female	51	Clothing	3	50
18	35	2023-08-05	CUST035	Female	58	Beauty	3	300
19	36	2023-06-24	CUST036	Male	52	Beauty	3	300
20	40	2023-06-22	CUST040	Male	45	Beauty	1	50

CODE

```
--Q6. Display all transactions where the Price per Unit is between 100 and 500.

--Expected output: All columns

SELECT *

FROM practical1.dataset.retail_sales

WHERE Price_per_Unit BETWEEN 100 AND 500;
```

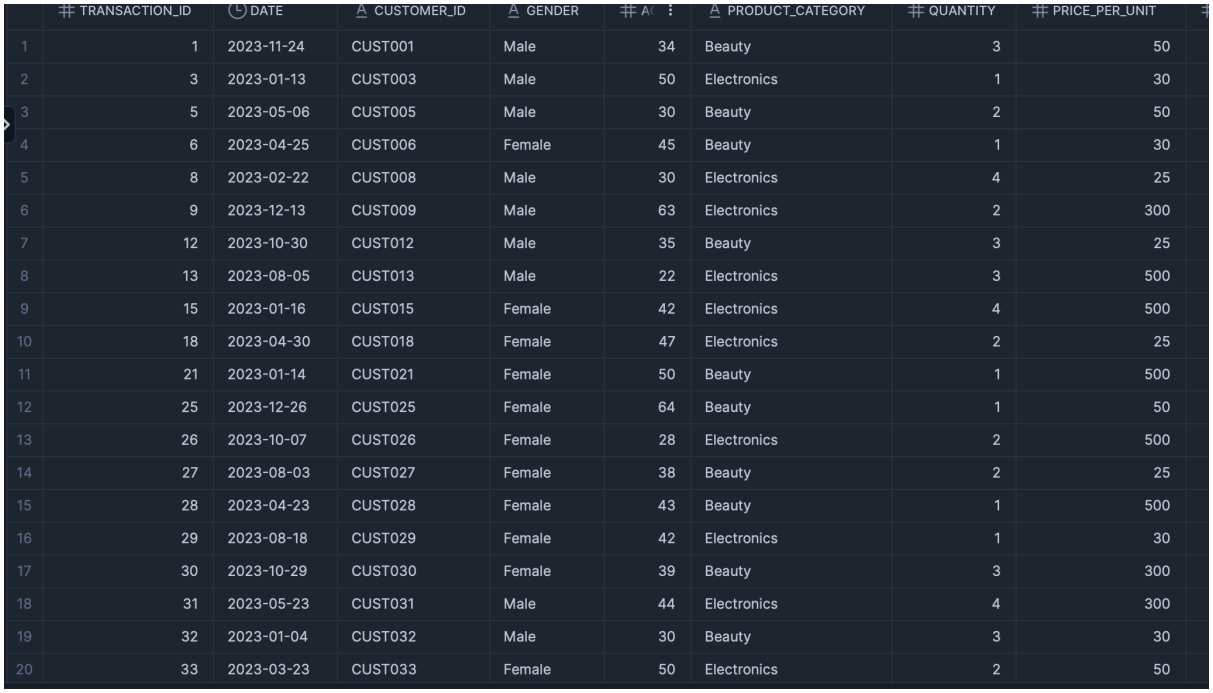
OUTPUT

	TRANSACTION_ID	DATE	CUSTOMER_ID	GENDER	AGE	PRODUCT_CATEGORY	QUANTITY	PRICE_PER_UNIT
1	2	2023-02-27	CUST002	Female	26	Clothing	2	500
2	4	2023-05-21	CUST004	Male	37	Clothing	1	500
3	9	2023-12-13	CUST009	Male	63	Electronics	2	300
4	13	2023-08-05	CUST013	Male	22	Electronics	3	500
5	15	2023-01-16	CUST015	Female	42	Electronics	4	500
6	16	2023-02-17	CUST016	Male	19	Clothing	3	500
7	20	2023-11-05	CUST020	Male	22	Clothing	3	300
8	21	2023-01-14	CUST021	Female	50	Beauty	1	500
9	24	2023-11-29	CUST024	Female	49	Clothing	1	300
10	26	2023-10-07	CUST026	Female	28	Electronics	2	500
11	28	2023-04-23	CUST028	Female	43	Beauty	1	500
12	30	2023-10-29	CUST030	Female	39	Beauty	3	300
13	31	2023-05-23	CUST031	Male	44	Electronics	4	300
14	35	2023-08-05	CUST035	Female	58	Beauty	3	300
15	36	2023-06-24	CUST036	Male	52	Beauty	3	300
16	42	2023-02-17	CUST042	Male	22	Clothing	3	300
17	43	2023-07-14	CUST043	Female	48	Clothing	1	300
18	46	2023-06-26	CUST046	Female	20	Electronics	4	300
19	47	2023-11-06	CUST047	Female	40	Beauty	3	500
20	48	2023-05-16	CUST048	Male	54	Electronics	3	300

CODE

```
--Q7. Display all transactions where the Product Category is either 'Beauty' or

--'Electronics'.

SELECT *

FROM practical1.dataset.retail_sales

WHERE Product_Category IN ('Beauty', 'Electronics');
```

OUTPUT

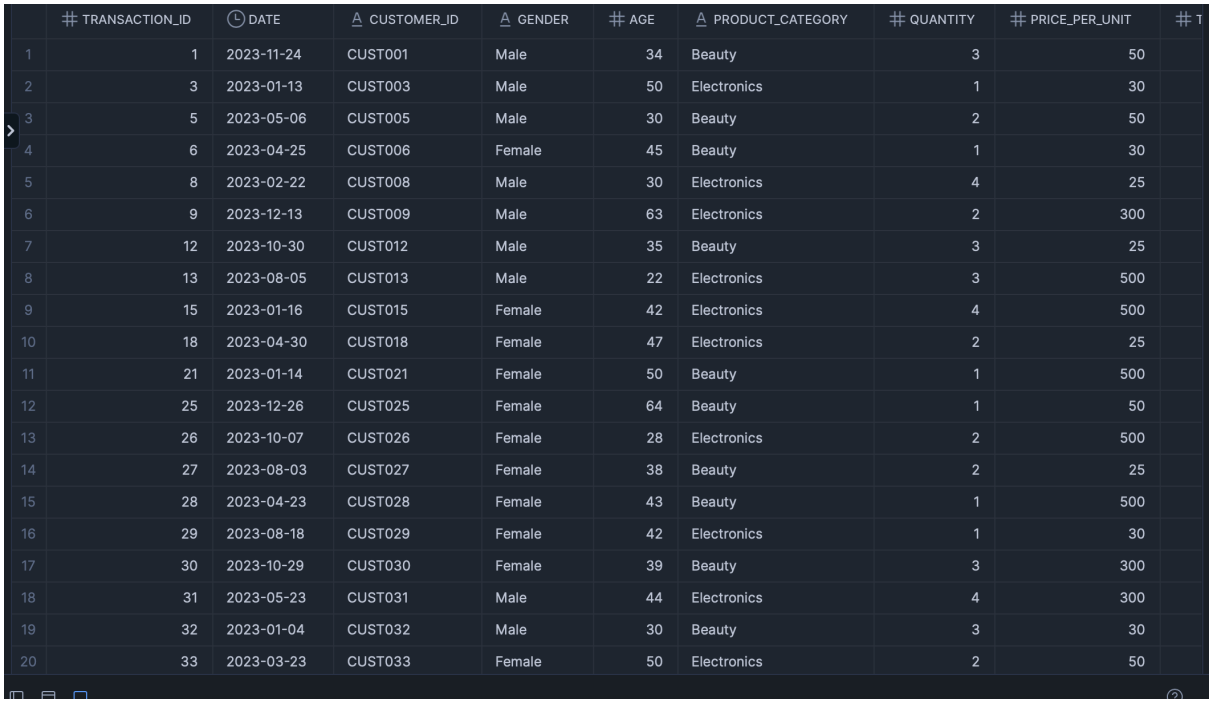


	TRANSACTION_ID	DATE	CUSTOMER_ID	GENDER	A	PRODUCT_CATEGORY	QUANTITY	PRICE_PER_UNIT
1	1	2023-11-24	CUST001	Male	34	Beauty	3	50
2	3	2023-01-13	CUST003	Male	50	Electronics	1	30
3	5	2023-05-06	CUST005	Male	30	Beauty	2	50
4	6	2023-04-25	CUST006	Female	45	Beauty	1	30
5	8	2023-02-22	CUST008	Male	30	Electronics	4	25
6	9	2023-12-13	CUST009	Male	63	Electronics	2	300
7	12	2023-10-30	CUST012	Male	35	Beauty	3	25
8	13	2023-08-05	CUST013	Male	22	Electronics	3	500
9	15	2023-01-16	CUST015	Female	42	Electronics	4	500
10	18	2023-04-30	CUST018	Female	47	Electronics	2	25
11	21	2023-01-14	CUST021	Female	50	Beauty	1	500
12	25	2023-12-26	CUST025	Female	64	Beauty	1	50
13	26	2023-10-07	CUST026	Female	28	Electronics	2	500
14	27	2023-08-03	CUST027	Female	38	Beauty	2	25
15	28	2023-04-23	CUST028	Female	43	Beauty	1	500
16	29	2023-08-18	CUST029	Female	42	Electronics	1	30
17	30	2023-10-29	CUST030	Female	39	Beauty	3	300
18	31	2023-05-23	CUST031	Male	44	Electronics	4	300
19	32	2023-01-04	CUST032	Male	30	Beauty	3	30
20	33	2023-03-23	CUST033	Female	50	Electronics	2	50

CODE

```
--Q8. Display all transactions where the Product Category is not 'Clothing'.

--Expected output: All columns

SELECT *

FROM practical1.dataset.retail_sales

WHERE Product_Category <> 'Clothing';
```

OUTPUT



	TRANSACTION_ID	DATE	CUSTOMER_ID	GENDER	AGE	PRODUCT_CATEGORY	QUANTITY	PRICE_PER_UNIT	T
1	1	2023-11-24	CUST001	Male	34	Beauty	3	50	
2	3	2023-01-13	CUST003	Male	50	Electronics	1	30	
3	5	2023-05-06	CUST005	Male	30	Beauty	2	50	
4	6	2023-04-25	CUST006	Female	45	Beauty	1	30	
5	8	2023-02-22	CUST008	Male	30	Electronics	4	25	
6	9	2023-12-13	CUST009	Male	63	Electronics	2	300	
7	12	2023-10-30	CUST012	Male	35	Beauty	3	25	
8	13	2023-08-05	CUST013	Male	22	Electronics	3	500	
9	15	2023-01-16	CUST015	Female	42	Electronics	4	500	
10	18	2023-04-30	CUST018	Female	47	Electronics	2	25	
11	21	2023-01-14	CUST021	Female	50	Beauty	1	500	
12	25	2023-12-26	CUST025	Female	64	Beauty	1	50	
13	26	2023-10-07	CUST026	Female	28	Electronics	2	500	
14	27	2023-08-03	CUST027	Female	38	Beauty	2	25	
15	28	2023-04-23	CUST028	Female	43	Beauty	1	500	
16	29	2023-08-18	CUST029	Female	42	Electronics	1	30	
17	30	2023-10-29	CUST030	Female	39	Beauty	3	300	
18	31	2023-05-23	CUST031	Male	44	Electronics	4	300	
19	32	2023-01-04	CUST032	Male	30	Beauty	3	30	
20	33	2023-03-23	CUST033	Female	50	Electronics	2	50	

CODE

```
--Q9. Display all transactions where the Quantity is greater than or equal to 3.

--Expected output: All columns

SELECT *

FROM practical1.dataset.retail_sales

WHERE Quantity >= 3;
```

OUTPUT

	ON_ID	DATE	CUSTOMER_ID	GENDER	AGE	PRODUCT_CATEGORY	QUANTITY	PRICE_PER_UNIT	TOTAL_AMOUNT
1	1	2023-11-24	CUST001	Male	34	Beauty	3	50	150
2	8	2023-02-22	CUST008	Male	30	Electronics	4	25	100
3	10	2023-10-07	CUST010	Female	52	Clothing	4	50	200
4	12	2023-10-30	CUST012	Male	35	Beauty	3	25	75
5	13	2023-08-05	CUST013	Male	22	Electronics	3	500	1500
6	14	2023-01-17	CUST014	Male	64	Clothing	4	30	120
7	15	2023-01-16	CUST015	Female	42	Electronics	4	500	2000
8	16	2023-02-17	CUST016	Male	19	Clothing	3	500	1500
9	17	2023-04-22	CUST017	Female	27	Clothing	4	25	100
10	20	2023-11-05	CUST020	Male	22	Clothing	3	300	900
11	23	2023-04-12	CUST023	Female	35	Clothing	4	30	120
12	30	2023-10-29	CUST030	Female	39	Beauty	3	300	900
13	31	2023-05-23	CUST031	Male	44	Electronics	4	300	1200
14	32	2023-01-04	CUST032	Male	30	Beauty	3	30	90
15	34	2023-12-24	CUST034	Female	51	Clothing	3	50	150
16	35	2023-08-05	CUST035	Female	58	Beauty	3	300	900
17	36	2023-06-24	CUST036	Male	52	Beauty	3	300	900
18	37	2023-05-23	CUST037	Female	18	Beauty	3	25	75
19	38	2023-03-21	CUST038	Male	38	Beauty	4	50	200
20	39	2023-04-21	CUST039	Male	23	Clothing	4	30	120

CODE

```
--4. Aggregate Functions

--Q10. Count the total number of transactions.

--Expected output: Total_Transactions

SELECT COUNT(*) AS Total_Transactions

FROM practical1.dataset.retail_sales;
```

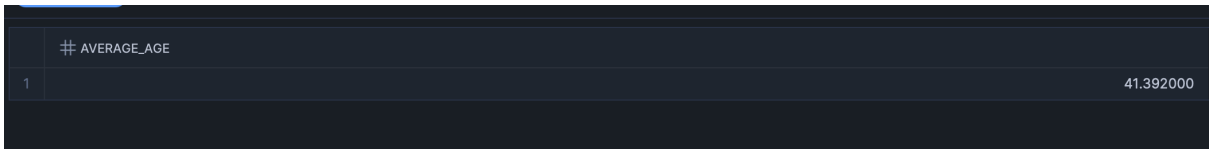
OUTPUT

	TOTAL_TRANSACTIONS
1	1000

CODE

```
--Q11. Find the average Age of customers.

--Expected output: Average_Age

SELECT AVG(Age) AS Average_Age

FROM practical1.dataset.retail_sales;
```

OUTPUT

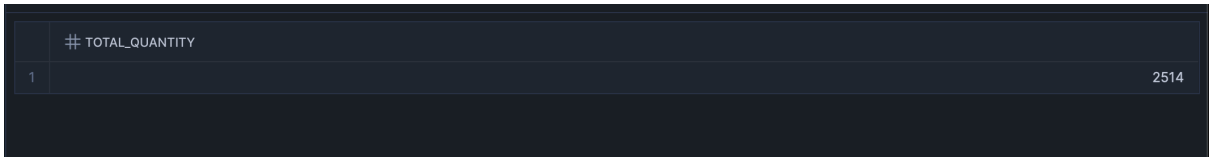


	AVERAGE_AGE
1	41.392000

CODE

```
--Q12. Find the total quantity of products sold.

--Expected output: Total_Quantity

SELECT SUM(Quantity) AS Total_Quantity

FROM practical1.dataset.retail_sales;
```

OUTPUT

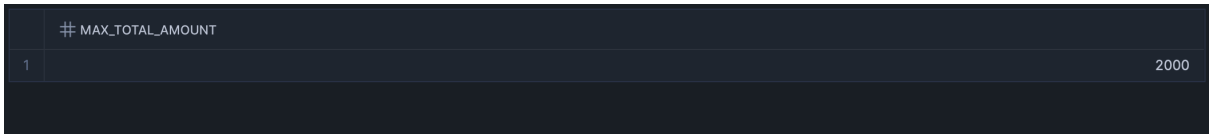


	TOTAL_QUANTITY
1	2514

CODE

```
--Q13. Find the maximum Total Amount spent in a single transaction.

--Expected output: Max_Total_Amount

SELECT MAX(Total_Amount) AS Max_Total_Amount

FROM practical1.dataset.retail_sales;
```

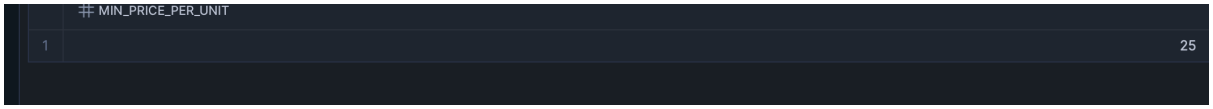
OUTPUT



CODE

```
--Expected output: Max_Total_Amount

--Q14. Find the minimum Price per Unit in the dataset.

--Expected output: Min_Price_per_Unit

SELECT MIN(Price_per_Unit) AS Min_Price_per_Unit

FROM practical1.dataset.retail_sales;
```

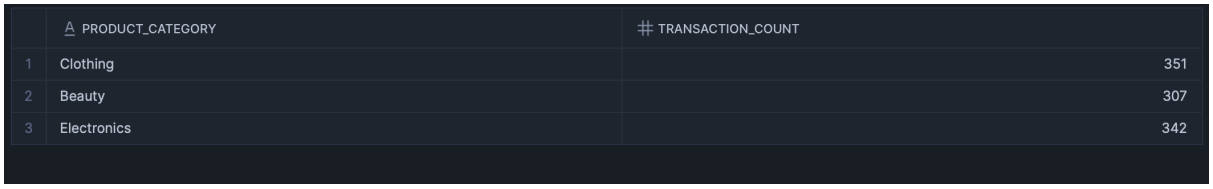
OUTPUT



CODE

```
--5. GROUP BY Statement

--Q15. Find the number of transactions per Product Category.

--Expected output: Product Category, Transaction_Count

SELECT

  Product_Category,

  COUNT(Transaction_ID) AS Transaction_Count

FROM practical1.dataset.retail_sales

GROUP BY Product_Category;
```

OUTPUT

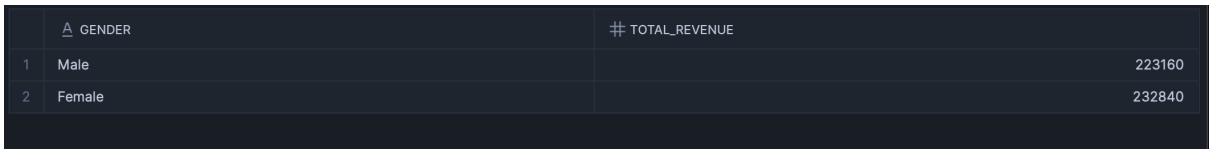


	PRODUCT_CATEGORY	TRANSACTION_COUNT
1	Clothing	351
2	Beauty	307
3	Electronics	342

CODE

```
--Q16. Find the total revenue (Total Amount) per gender.
--Expected output: Gender, Total_Revenue
SELECT
  Gender,
  SUM(Total_Amount) AS Total_Revenue
FROM practical1.dataset.retail_sales
GROUP BY Gender;
```

OUTPUT

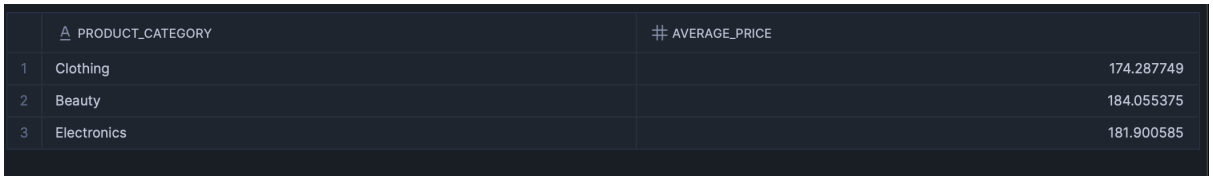


	GENDER	TOTAL_REVENUE
1	Male	223160
2	Female	232840

CODE

```
--Q17. Find the average Price per Unit per product category.
--Expected output: Product Category, Average_Price
SELECT
  Product_Category,
  AVG(Price_per_Unit) AS Average_Price
FROM practical1.dataset.retail_sales
GROUP BY Product_Category;
```

OUTPUT

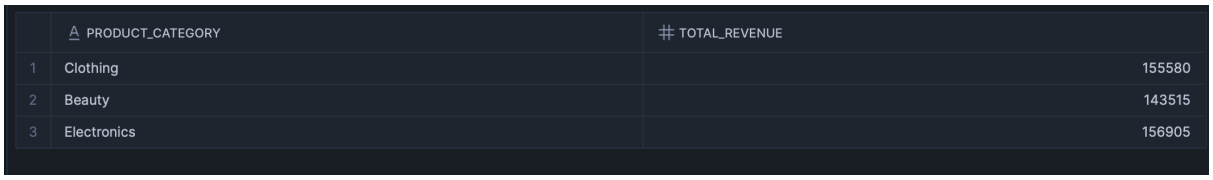


	PRODUCT_CATEGORY	AVERAGE_PRICE
1	Clothing	174.287749
2	Beauty	184.055375
3	Electronics	181.900585

CODE

```
--6. HAVING Clause
--Q18. Find the total revenue per product category where total revenue is greater than
--10,000.
--Expected output: Product Category, Total_Revenue
SELECT
  Product_Category,
  SUM(Total_Amount) AS Total_Revenue
FROM practical1.dataset.retail_sales
GROUP BY Product_Category
HAVING SUM(Total_Amount) > 10000;
```

OUTPUT



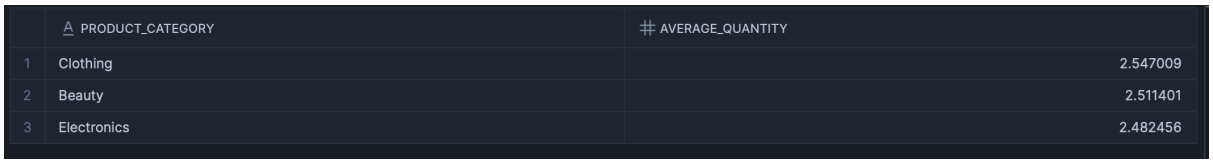
	PRODUCT_CATEGORY	TOTAL_REVENUE
1	Clothing	155580
2	Beauty	143515
3	Electronics	156905

CODE

```
--Q19. Find the average quantity per product category where the average is more than 2.
--Expected output: Product Category, Average_Quantity
SELECT
```

```
  Product_Category,
  AVG(Quantity) AS Average_Quantity
FROM practical1.dataset.retail_sales
GROUP BY Product_Category
HAVING AVG(Quantity) > 2;
```

OUTPUT

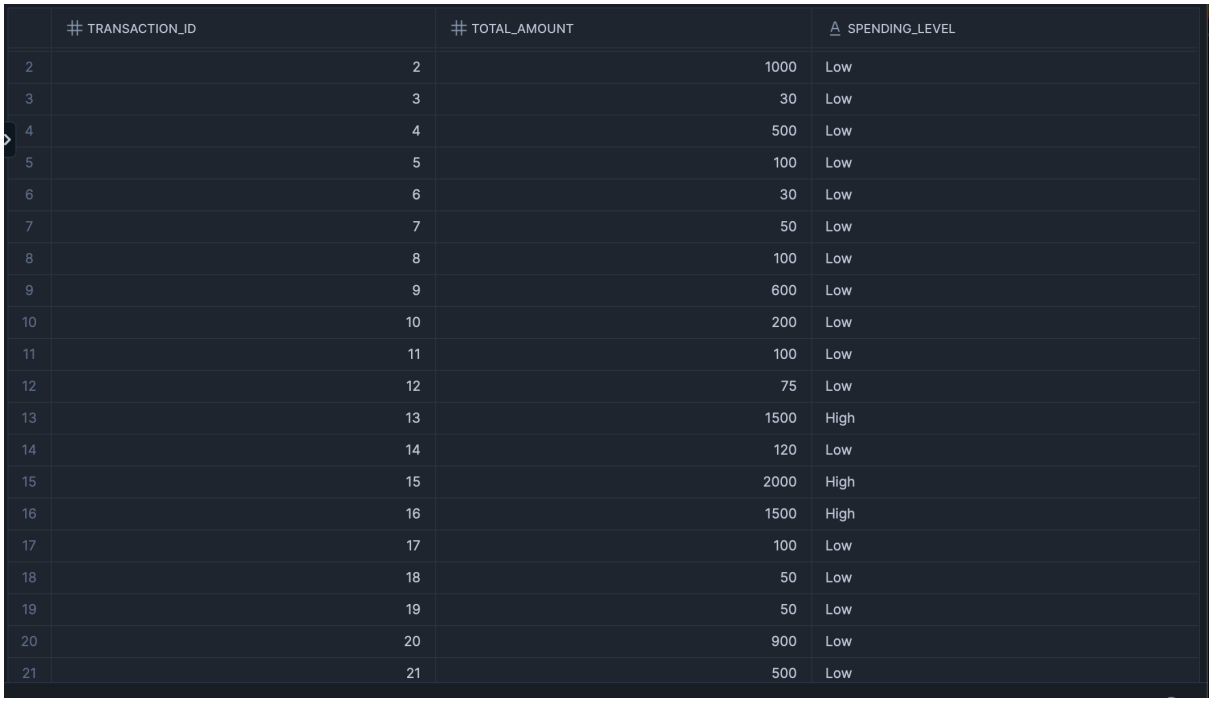


	PRODUCT_CATEGORY	AVERAGE_QUANTITY
1	Clothing	2.547009
2	Beauty	2.511401
3	Electronics	2.482456

CODE

```
--7. CASE Statement
--Q20. Display a column called Spending_Level that shows 'High' if Total Amount > 1000,
--otherwise 'Low'.
--Expected output: Transaction ID, Total Amount, Spending_Level
SELECT
  Transaction_ID,
  Total_Amount,
  CASE
    WHEN Total_Amount > 1000 THEN 'High'
    ELSE 'Low'
  END AS Spending_Level
FROM practical1.dataset.retail_sales;
```

OUTPUT



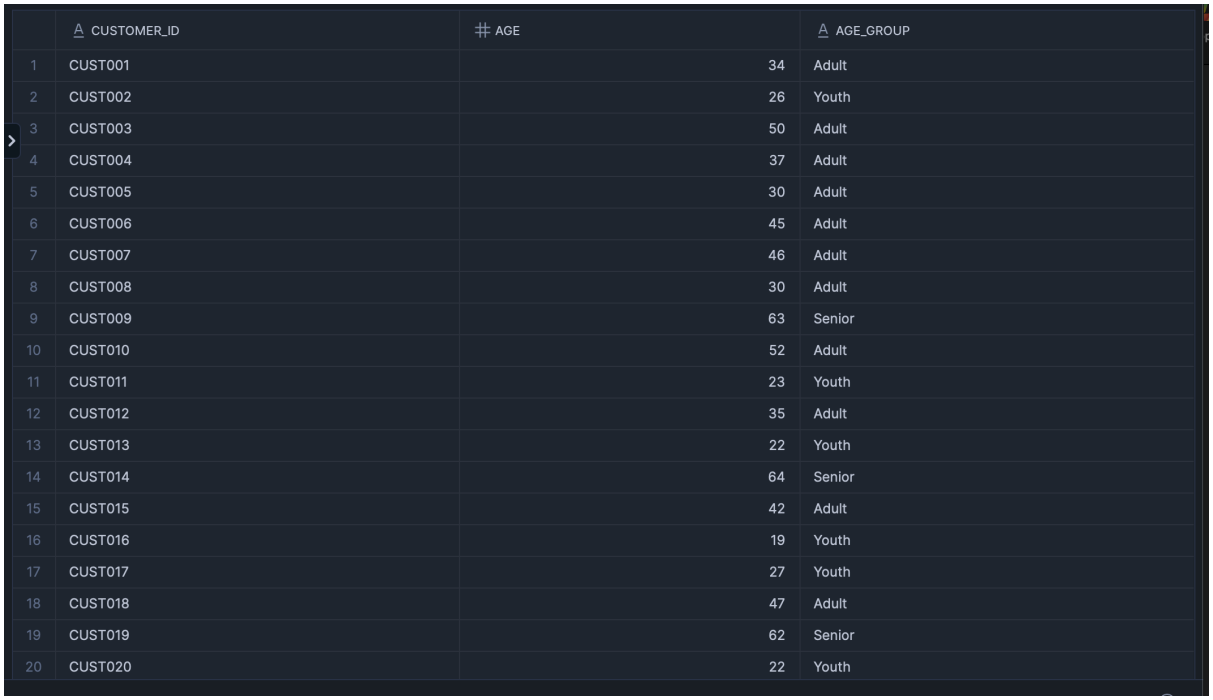
	TRANSACTION_ID	TOTAL_AMOUNT	SPENDING_LEVEL
2	2	1000	Low
3	3	30	Low
4	4	500	Low
5	5	100	Low
6	6	30	Low
7	7	50	Low
8	8	100	Low
9	9	600	Low
10	10	200	Low
11	11	100	Low
12	12	75	Low
13	13	1500	High
14	14	120	Low
15	15	2000	High
16	16	1500	High
17	17	100	Low
18	18	50	Low
19	19	50	Low
20	20	900	Low
21	21	500	Low

CODE

```
--Q21. Display a new column called Age_Group that labels customers as:
--• 'Youth' if Age < 30
--• 'Adult' if Age is between 30 and 59
--• 'Senior' if Age >= 60
--Expected output: Customer ID, Age, Age_Group
SELECT
  Customer_ID,
  Age,
  CASE
    WHEN Age < 30 THEN 'Youth'
    WHEN Age BETWEEN 30 AND 59 THEN 'Adult'
    ELSE 'Senior'
  END AS Age_Group
```

```
FROM practical1.dataset.retail_sales;
```

OUTPUT



	CUSTOMER_ID	AGE	AGE_GROUP
1	CUST001	34	Adult
2	CUST002	26	Youth
3	CUST003	50	Adult
4	CUST004	37	Adult
5	CUST005	30	Adult
6	CUST006	45	Adult
7	CUST007	46	Adult
8	CUST008	30	Adult
9	CUST009	63	Senior
10	CUST010	52	Adult
11	CUST011	23	Youth
12	CUST012	35	Adult
13	CUST013	22	Youth
14	CUST014	64	Senior
15	CUST015	42	Adult
16	CUST016	19	Youth
17	CUST017	27	Youth
18	CUST018	47	Adult
19	CUST019	62	Senior
20	CUST020	22	Youth

ALL CODE

```
-SELECT

--Q1. Display all columns for all transactions.

--Expected output: All columns

SELECT*

FROM practical1.dataset.retail_sales;


--Q2. Display only the Transaction ID, Date, and Customer ID for all records.

--Expected output: Transaction ID, Date, Customer ID

SELECT transaction_id,

    date,

    customer_id

FROM practical1.dataset.retail_sales;
```

```
--2. SELECT DISTINCT Statement

--Q3. Display all the distinct product categories in the dataset.

--Expected output: Product Category

SELECT DISTINCT product_category

FROM practical1.dataset.retail_sales;



--Q4. Display all the distinct gender values in the dataset.

--Expected output: Gender

SELECT DISTINCT Gender

FROM practical1.dataset.retail_sales;



--3. WHERE Clause

--Q5. Display all transactions where the Age is greater than 40.

--Expected output: All columns

SELECT *

FROM practical1.dataset.retail_sales

WHERE Age > 40;



--Q6. Display all transactions where the Price per Unit is between 100 and 500.

--Expected output: All columns

SELECT *

FROM practical1.dataset.retail_sales

WHERE Price_per_Unit BETWEEN 100 AND 500;



--Q7. Display all transactions where the Product Category is either 'Beauty' or

--'Electronics'.
```

```
SELECT *
FROM practical1.dataset.retail_sales
WHERE Product_Category IN ('Beauty', 'Electronics');

--Q8. Display all transactions where the Product Category is not 'Clothing'.
--Expected output: All columns
SELECT *
FROM practical1.dataset.retail_sales
WHERE Product_Category <> 'Clothing';

--Q9. Display all transactions where the Quantity is greater than or equal to 3.
--Expected output: All columns
SELECT *
FROM practical1.dataset.retail_sales
WHERE Quantity >= 3;


--4. Aggregate Functions
--Q10. Count the total number of transactions.
--Expected output: Total_Transactions
SELECT COUNT(*) AS Total_Transactions
FROM practical1.dataset.retail_sales

--Q11. Find the average Age of customers.
--Expected output: Average_Age
SELECT AVG(Age) AS Average_Age
FROM practical1.dataset.retail_sales;
```

```
--Q12. Find the total quantity of products sold.
--Expected output: Total_Quantity
SELECT SUM(Quantity) AS Total_Quantity
FROM practical1.dataset.retail_sales;

--Q13. Find the maximum Total Amount spent in a single transaction.
--Expected output: Max_Total_Amount
SELECT MAX(Total_Amount) AS Max_Total_Amount
FROM practical1.dataset.retail_sales;

--Expected output: Max_Total_Amount
--Q14. Find the minimum Price per Unit in the dataset.
--Expected output: Min_Price_per_Unit
SELECT MIN(Price_per_Unit) AS Min_Price_per_Unit
FROM practical1.dataset.retail_sales;


--5. GROUP BY Statement
--Q15. Find the number of transactions per Product Category.
--Expected output: Product Category, Transaction_Count
SELECT
  Product_Category,
  COUNT(Transaction_ID) AS Transaction_Count
FROM practical1.dataset.retail_sales
GROUP BY Product_Category;

--Q16. Find the total revenue (Total Amount) per gender.
--Expected output: Gender, Total_Revenue
```

```
SELECT
  Gender,
  SUM(Total_Amount) AS Total_Revenue
FROM practical1.dataset.retail_sales
GROUP BY Gender;

--Q17. Find the average Price per Unit per product category.
--Expected output: Product Category, Average_Price
SELECT
  Product_Category,
  AVG(Price_per_Unit) AS Average_Price
FROM practical1.dataset.retail_sales
GROUP BY Product_Category;



--6. HAVING Clause
--Q18. Find the total revenue per product category where total revenue is greater than
--10,000.
--Expected output: Product Category, Total_Revenue
SELECT
  Product_Category,
  SUM(Total_Amount) AS Total_Revenue
FROM practical1.dataset.retail_sales
GROUP BY Product_Category
HAVING SUM(Total_Amount) > 10000;

--Q19. Find the average quantity per product category where the average is more than 2.
--Expected output: Product Category, Average_Quantity
```

```
SELECT
  Product_Category,
  AVG(Quantity) AS Average_Quantity
FROM practical1.dataset.retail_sales
GROUP BY Product_Category
HAVING AVG(Quantity) > 2;



--7. CASE Statement
--Q20. Display a column called Spending_Level that shows 'High' if Total Amount > 1000,
--otherwise 'Low'.
--Expected output: Transaction ID, Total Amount, Spending_Level
SELECT
  Transaction_ID,
  Total_Amount,
  CASE
    WHEN Total_Amount > 1000 THEN 'High'
    ELSE 'Low'
  END AS Spending_Level
FROM practical1.dataset.retail_sales;



--Q21. Display a new column called Age_Group that labels customers as:
--• 'Youth' if Age < 30
--• 'Adult' if Age is between 30 and 59
--• 'Senior' if Age >= 60
--Expected output: Customer ID, Age, Age_Group
```

```
SELECT
  Customer_ID,
  Age,
  CASE
    WHEN Age < 30 THEN 'Youth'
    WHEN Age BETWEEN 30 AND 59 THEN 'Adult'
    ELSE 'Senior'
  END AS Age_Group
FROM practical1.dataset.retail_sales;
```